

Universidad Privada Domingo Savio

Tecnología Web II

Entrega 10B

Nota Final Ponderada, Dashboards y Cierre Fase 4

FASE 4: Horarios, Asistencias y Configuración de Evaluación

Abril 2026

Objetivos de Aprendizaje

- Implementar la función de cálculo de nota final ponderada que combina calificaciones con ponderaciones de evaluación.
- Crear un API Route que devuelve las notas finales de todos los estudiantes de una materia.
- Construir un componente de tabla de notas finales con indicadores de aprobación/reprobación.
- Integrar tarjetas de estadísticas de Fase 4 en los dashboards existentes de admin, docente y estudiante.
- Completar el checklist de cierre de Fase 4 y documentar el estado final del sistema.

Prerrequisitos

- ✓ Entregas 8A a 10A completadas: tablas horarios, asistencias y configuracion_evaluacion funcionales.
- ✓ Al menos 1 materia con configuración de evaluación completa (suma = 100%).
- ✓ Al menos 2 estudiantes con calificaciones registradas en esa materia (Entrega 6A-6B).
- ✓ Componentes de dashboards existentes desde Entregas 4A-7B.

Teoria: Calculo de Nota Final Ponderada

La nota final ponderada combina las calificaciones individuales del estudiante con los pesos definidos en la configuracion de evaluacion. Es el resultado mas importante del sistema academico porque determina si el estudiante aprueba o reprueba.

Formula

Formula de Nota Final Ponderada

```
Nota_Final = SUM( calificacion_i * (peso_i / 100) )
```

Donde:

calificacion_i = nota del estudiante en tipo_evaluacion i (0-100)

peso_i = porcentaje asignado a ese tipo (de configuracion_evaluacion)

Ejemplo concreto:

Tipo	Peso	Calificacion	Aporte
parcial_1	25%	78	78 * 0.25 = 19.5
parcial_2	25%	85	85 * 0.25 = 21.25
practica	25%	90	90 * 0.25 = 22.5
proyecto	25%	72	72 * 0.25 = 18.0
TOTAL	100%		81.25

Resultado: 81.25 -> APROBADO (minimo 51 en Bolivia)

Casos especiales:

- Si falta una calificacion obligatoria -> 'Incompleto'
- Si la config no suma 100% -> 'Config incompleta'
- Tipo 'extra' suma al total pero no es obligatorio

Rango de Nota	Estado	Color en UI
90 - 100	Excelente	Verde oscuro
70 - 89	Bueno	Verde claro
51 - 69	Aprobado	Amarillo
0 - 50	Reprobado	Rojo
Sin datos	Incompleto	Gris

NOTA: Nota minima de aprobacion en Bolivia

En el sistema universitario boliviano, la nota minima de aprobacion es 51 sobre 100 (o su equivalente 3.1 sobre 7 en algunas universidades). Nuestro sistema usa escala de 0-100 por consistencia con la tabla calificaciones del MVP.

Paso 1: Funcion de Calculo de Nota Final

Crea una funcion utilitaria pura (sin dependencias de BD ni React) que recibe calificaciones y configuraciones y devuelve la nota final calculada.

```
TypeScript - lib/utils/calcular-nota-final.ts (crear archivo nuevo)
// — Tipos —
interface ConfigEvaluacion {
  tipo_evaluacion: string;
  peso: number;
  obligatoria: boolean;
}

interface Calificacion {
  tipo_evaluacion: string;
  nota: number;
}

interface ResultadoNotaFinal {
  notaFinal: number | null;
  estado: "excelente" | "bueno" | "aprobado" | "reprobado" | "incompleto" | "sin_config";
  desglose: {
    tipo: string;
    peso: number;
    nota: number | null;
    aporte: number;
    obligatoria: boolean;
  }[];
  mensaje: string;
  faltantes: string[];
}

// — Funcion principal —
export function calcularNotaFinal(
  configuraciones: ConfigEvaluacion[],
  calificaciones: Calificacion[]
): ResultadoNotaFinal {
  // Sin configuracion
  if (configuraciones.length === 0) {
    return {
      notaFinal: null,
      estado: "sin_config",
      desglose: [],
      mensaje: "Sin configuracion de evaluacion",
      faltantes: [],
    };
  }
}
```

```

// Verificar que la config sume 100
const sumaConfig = configuraciones
  .filter((c) => c.tipo_evaluacion !== "extra")
  .reduce((acc, c) => acc + c.peso, 0);

const configCompleta = Math.abs(sumaConfig - 100) < 0.01;

// Crear mapa de calificaciones
const notasMap = new Map<string, number>();
calificaciones.forEach((cal) => {
  notasMap.set(cal.tipo_evaluacion, cal.nota);
});

// Calcular desglose
const faltantes: string[] = [];
let notaAcumulada = 0;

const desglose = configuraciones.map((cfg) => {
  const nota = notasMap.get(cfg.tipo_evaluacion) ?? null;
  const aporte = nota !== null ? nota * (cfg.peso / 100) : 0;

  if (nota === null && cfg.obligatoria) {
    faltantes.push(cfg.tipo_evaluacion);
  }

  if (nota !== null) {
    notaAcumulada += aporte;
  }

  return {
    tipo: cfg.tipo_evaluacion,
    peso: cfg.peso,
    nota,
    aporte: Math.round(aporte * 100) / 100,
    obligatoria: cfg.obligatoria,
  };
});

// Determinar estado
if (!configCompleta) {
  return {
    notaFinal: null,
    estado: "sin_config",
    desglose,
    mensaje: "Configuracion de evaluacion incompleta",
    faltantes,
  };
}

if (faltantes.length > 0) {

```

```

    return {
      notaFinal: Math.round(notaAcumulada * 100) / 100,
      estado: "incompleto",
      desglose,
      mensaje: `Faltan ${faltantes.length} calificacion(es) obligatoria(s)`,
      faltantes,
    };
  }

  const notaFinal = Math.round(notaAcumulada * 100) / 100;

  let estado: ResultadoNotaFinal["estado"];
  if (notaFinal >= 90) estado = "excelente";
  else if (notaFinal >= 70) estado = "bueno";
  else if (notaFinal >= 51) estado = "aprobado";
  else estado = "reprobado";

  return {
    notaFinal,
    estado,
    desglose,
    mensaje: estado === "reprobado"
      ? "Reprobado (nota < 51)"
      : `Aprobado (${notaFinal})`,
    faltantes: [],
  };
}

// — Colores para UI —
export const COLORES_ESTADO: Record<string, string> = {
  excelente: "bg-emerald-100 text-emerald-800",
  bueno: "bg-green-100 text-green-800",
  aprobado: "bg-yellow-100 text-yellow-800",
  reprobado: "bg-red-100 text-red-800",
  incompleto: "bg-gray-100 text-gray-600",
  sin_config: "bg-gray-50 text-gray-400",
};

export const LABELS_ESTADO: Record<string, string> = {
  excelente: "Excelente",
  bueno: "Bueno",
  aprobado: "Aprobado",
  reprobado: "Reprobado",
  incompleto: "Incompleto",
  sin_config: "Sin Config",
};

```

TIP: Funcion pura sin side effects

calcularNotaFinal es una funcion pura: recibe datos, devuelve resultado. No accede a la BD, no modifica estado global, no llama APIs. Esto la hace facil de testear con unit tests y reutilizable tanto en API Routes como en componentes frontend.

Paso 2: API Route Docente - Notas Finales

Este endpoint calcula las notas finales de todos los estudiantes de una materia del docente. Combina datos de 3 tablas: inscripciones, calificaciones y configuracion_evaluacion.

```
TypeScript - app/api/docente/notas-finales/route.ts (crear archivo nuevo)
import { createClient } from "@lib/supabase/server";
import { NextRequest, NextResponse } from "next/server";
import { calcularNotaFinal } from "@lib/utils/calcular-nota-final";

export async function GET(request: NextRequest) {
  const supabase = await createClient();

  const { data: { user }, error: authError } = await supabase.auth.getUser();
  if (authError || !user) {
    return NextResponse.json({ error: "No autenticado" }, { status: 401 });
  }

  const { searchParams } = new URL(request.url);
  const materia_id = searchParams.get("materia_id");

  if (!materia_id) {
    return NextResponse.json({ error: "materia_id requerido" }, { status: 400 });
  }

  // Verificar que la materia pertenece al docente
  const { data: materia } = await supabase
    .from("materias")
    .select("id, nombre, codigo")
    .eq("id", materia_id)
    .eq("docente_id", user.id)
    .single();

  if (!materia) {
    return NextResponse.json({ error: "Acceso denegado" }, { status: 403 });
  }

  // Obtener configuracion de evaluacion
  const { data: configs } = await supabase
    .from("configuracion_evaluacion")
    .select("tipo_evaluacion, peso, obligatoria")
    .eq("materia_id", materia_id)
    .order("tipo_evaluacion");

  // Obtener inscripciones con perfil
  const { data: inscripciones } = await supabase
    .from("inscripciones")
```

```

        .select(`
            id,
            estudiante_id,
            usuarios_perfil!inscripciones_estudiante_id_fkey (
                id, nombre, apellido, email
            )
        `)
        .eq("materia_id", materia_id)
        .eq("estado", "inscrito");

if (!inscripciones || inscripciones.length === 0) {
    return NextResponse.json({
        materia,
        configuraciones: configs || [],
        estudiantes: [],
        resumen: { total: 0, aprobados: 0, reprobados: 0, incompletos: 0 },
    });
}

// Obtener todas las calificaciones de la materia
const { data: calificaciones } = await supabase
    .from("calificaciones")
    .select("inscripcion_id, tipo_evaluacion, nota")
    .in("inscripcion_id", inscripciones.map((i) => i.id));

// Calcular nota final por estudiante
let aprobados = 0;
let reprobados = 0;
let incompletos = 0;

const estudiantes = inscripciones.map((insc) => {
    const calEstudiante = (calificaciones || []).filter(
        (cal) => cal.inscripcion_id === insc.id
    );
});

const resultado = calcularNotaFinal(
    configs || [],
    calEstudiante
);

if (resultado.estado === "incompleto" || resultado.estado === "sin_config") {
    incompletos++;
} else if (resultado.estado === "reprobado") {
    reprobados++;
} else {
    aprobados++;
}

return {
    estudiante: insc.usuarios_perfil,

```

```

        inscripcion_id: insc.id,
        resultado,
    };
});

// Ordenar por nota final (descendente)
estudiantes.sort((a, b) => {
    const notaA = a.resultado.notaFinal ?? -1;
    const notaB = b.resultado.notaFinal ?? -1;
    return notaB - notaA;
});

return NextResponse.json({
    materia,
    configuraciones: configs || [],
    estudiantes,
    resumen: {
        total: inscripciones.length,
        aprobados,
        reprobados,
        incompletos,
        promedioGeneral: estudiantes.length > 0
            ? Math.round(
                estudiantes
                    .filter((e) => e.resultado.notaFinal !== null)
                    .reduce((acc, e) => acc + (e.resultado.notaFinal || 0), 0)
                / Math.max(estudiantes.filter((e) => e.resultado.notaFinal !== null).length, 1)
                * 100) / 100
            : 0,
    },
});
}

```

NOTA: Consulta a 3 tablas en paralelo

Este endpoint consulta `configuracion_evaluacion`, `inscripciones` y `calificaciones` para una materia. Los datos se combinan en el servidor con la función `calcularNotaFinal()`. El resultado incluye un resumen con contadores de aprobados/reprobados para las tarjetas KPI.

Paso 3: Componente Notas Finales (Docente)

Tabla de notas finales con desglose expandible por estudiante, badges de estado y resumen con tarjetas KPI.

TypeScript - components/dashboard/docente/notas-finales-table.tsx (Parte 1)

```
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";
import {
  Trophy, ChevronDown, ChevronUp, Users,
  CheckCircle, XCircle, AlertTriangle,
} from "lucide-react";

import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Progress } from "@components/ui/progress";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import { Label } from "@components/ui/label";
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";

import {
  COLORES_ESTADO, LABELS_ESTADO
} from "@lib/utils/calcular-nota-final";
import { TIPOS_EVALUACION } from "@lib/validations/configuracion-evaluacion";

interface Materia { id: string; nombre: string; codigo: string; }
interface Desglose {
  tipo: string; peso: number; nota: number | null;
  aporte: number; obligatoria: boolean;
}
interface Resultado {
  notaFinal: number | null; estado: string;
  desglose: Desglose[]; mensaje: string; faltantes: string[];
}
interface EstudianteNota {
  estudiante: { id: string; nombre: string; apellido: string; email: string; };
  inscripcion_id: string;
  resultado: Resultado;
}
interface Resumen {
```

```

total: number; aprobados: number;
reprobados: number; incompletos: number; promedioGeneral: number;
}

```

TypeScript - notas-finales-table.tsx (Parte 2: Componente)

```

export default function NotasFinalesTable() {
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [materiaSeleccionada, setMateriaSeleccionada] = useState("");
  const [estudiantes, setEstudiantes] = useState<EstudianteNota[]>([]);
  const [resumen, setResumen] = useState<Resumen | null>(null);
  const [expandido, setExpandido] = useState<string | null>(null);
  const [cargando, setCargando] = useState(false);

  // Cargar materias
  useEffect(() => {
    async function cargar() {
      const res = await fetch("/api/docente/materias");
      if (res.ok) setMaterias(await res.json());
    }
    cargar();
  }, []);

  // Cargar notas al seleccionar materia
  useEffect(() => {
    if (!materiaSeleccionada) return;
    async function cargar() {
      setCargando(true);
      try {
        const params = new URLSearchParams({
          materia_id: materiaSeleccionada,
        });
        const res = await fetch(`/api/docente/notas-finales?${params}`);
        if (res.ok) {
          const data = await res.json();
          setEstudiantes(data.estudiantes);
          setResumen(data.resumen);
        }
      } catch {
        toast.error("Error al cargar notas");
      } finally { setCargando(false); }
    }
    cargar();
  }, [materiaSeleccionada]);

  function labelTipo(tipo: string) {
    return TIPOS_EVALUACION.find((t) => t.value === tipo)?.label || tipo;
  }
}

```

TypeScript - notas-finales-table.tsx (Parte 3: Render)

```

return (
  <div className="space-y-6">
    {/* Selector de materia */}
    <div className="grid gap-2 max-w-sm">
      <Label>Seleccionar Materia</Label>
      <Select value={materiaSeleccionada} onChange={setMateriaSeleccionada}>
        <SelectTrigger>
          <SelectValue placeholder="Seleccionar materia" />
        </SelectTrigger>
        <SelectContent>
          {materias.map((m) => (
            <SelectItem key={m.id} value={m.id}>
              {m.codigo} - {m.nombre}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>

    {cargando && <p className="text-muted-foreground">Calculando notas...</p>}}

    {/* Tarjetas KPI */}
    {resumen && !cargando && (
      <div className="grid grid-cols-2 md:grid-cols-5 gap-3">
        <Card><CardContent className="pt-4 text-center">
          <p className="text-2xl font-bold">{resumen.total}</p>
          <p className="text-xs text-muted-foreground">Estudiantes</p>
        </CardContent></Card>
        <Card><CardContent className="pt-4 text-center">
          <p className="text-2xl font-bold text-blue-600">{resumen.promedioGeneral}</p>
          <p className="text-xs text-muted-foreground">Promedio General</p>
        </CardContent></Card>
        <Card><CardContent className="pt-4 text-center">
          <p className="text-2xl font-bold text-green-600">{resumen.aprobados}</p>
          <p className="text-xs text-muted-foreground">Aprobados</p>
        </CardContent></Card>
        <Card className={resumen.reprobados > 0 ? "border-red-200" : ""}>
          <CardContent className="pt-4 text-center">
            <p className="text-2xl font-bold text-red-600">{resumen.reprobados}</p>
            <p className="text-xs text-muted-foreground">Reprobados</p>
          </CardContent></Card>
        <Card><CardContent className="pt-4 text-center">
          <p className="text-2xl font-bold text-gray-500">{resumen.incompletos}</p>
          <p className="text-xs text-muted-foreground">Incompletos</p>
        </CardContent></Card>
      </div>
    )}

    {/* Tabla de notas */}
    {estudiantes.length > 0 && !cargando && (
      <Card>

```

```

<CardHeader>
  <CardTitle className="flex items-center gap-2">
    <Trophy className="h-5 w-5" />
    Notas Finales
  </CardTitle>
</CardHeader>
<CardContent>
  <Table>
    <TableHeader><TableRow>
      <TableHead className="w-[40px]">#</TableHead>
      <TableHead>Estudiante</TableHead>
      <TableHead>Nota Final</TableHead>
      <TableHead>Estado</TableHead>
      <TableHead className="w-[40px]"></TableHead>
    </TableRow></TableHeader>
    <TableBody>
      {estudiantes.map((est, idx) => (
        <<TableRow key={est.inscripcion_id}
          className="cursor-pointer"
          onClick={() => setExpandido(
            expandido === est.inscripcion_id ? null : est.inscripcion_id
          )}
        >
          <TableCell>{idx + 1}</TableCell>
          <TableCell>
            <p className="font-medium">
              {est.estudiante.apellido}, {est.estudiante.nombre}
            </p>
            <p className="text-xs text-muted-foreground">
              {est.estudiante.email}
            </p>
          </TableCell>
          <TableCell>
            <span className="text-xl font-bold font-mono">
              {est.resultado.notaFinal !== null
                ? est.resultado.notaFinal
                : "-"}
            </span>
          </TableCell>
          <TableCell>
            <Badge className={COLORES_ESTADO[est.resultado.estado]}>
              {LABELS_ESTADO[est.resultado.estado]}
            </Badge>
          </TableCell>
          <TableCell>
            {expandido === est.inscripcion_id
              ? <ChevronUp className="h-4 w-4" />
              : <ChevronDown className="h-4 w-4" />}
          </TableCell>
        </TableRow>
      )}

      {/* Desglose expandible */}
      {expandido === est.inscripcion_id && (

```

```

<TableRow>
  <TableCell colSpan={5} className="bg-muted/50 p-4">
    <div className="grid grid-cols-2 md:grid-cols-4 gap-3">
      {est.resultado.desglose.map((d) => (
        <div key={d.tipo} className="p-3 bg-background
          rounded-md border">
          <p className="text-xs text-muted-foreground">
            {labelTipo(d.tipo)} ({d.peso}%)
          </p>
          <p className="text-lg font-bold font-mono">
            {d.nota !== null ? d.nota : "-"}
          </p>
          <p className="text-xs text-muted-foreground">
            Aporte: {d.aporte}
          </p>
        </div>
      )
    )}
    </div>
    {est.resultado.faltantes.length > 0 && (
      <p className="text-sm text-red-600 mt-2">
        Falta(n): {est.resultado.faltantes
          .map(labelTipo).join(", ")}
      </p>
    )}
  </TableCell>
</TableRow>
)}</>
)}}
</TableBody>
</Table>
</CardContent>
</Card>
)}
</div>
);
}

```

IMPORTANTE: Fragment (<>) para filas expandibles

Usamos React Fragment (<>...</>) para envolver la fila principal y la fila de desglose porque <TableBody> solo acepta <TableRow> como hijos directos. Sin el Fragment, React lanza un warning de nesting incorrecto.

Paso 4: Pagina de Notas Finales (Docente)

TypeScript - app/(dashboard)/docente/notas-finales/page.tsx (crear pagina nueva)

```
import NotasFinalesTable from
  "@components/dashboard/docente/notas-finales-table";

export default function NotasFinalesPage() {
  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-6">Notas Finales</h1>
      <NotasFinalesTable />
    </div>
  );
}
```

Agrega la entrada al sidebar del docente en lib/navigation.ts:

TypeScript - Agregar en lib/navigation.ts (seccion docente)

```
import { Trophy } from "lucide-react"; // agregar al import

// En el array de items del docente, agregar:
{
  title: "Notas Finales",
  url: "/docente/notas-finales",
  icon: Trophy,
},
```

Tambien crea el directorio: `mkdir -p app/(dashboard)/docente/notas-finales`

Paso 5: Integrar Metricas en Dashboards

Actualiza los dashboards principales de admin, docente y estudiante con tarjetas que resuman las metricas de la Fase 4.

5.1 - Dashboard Admin

TypeScript - Actualizar `app/(dashboard)/admin/page.tsx`

```
// Agregar imports
import StatsAsistenciaCard from
  "@components/dashboard/stats-asistencia-card";
import { BarChart3, Clock, UserCheck } from "lucide-react";

// Agregar seccion en el JSX despues de las tarjetas existentes:

{/* — Metricas Fase 4 — */}
<div className="mt-8">
  <h2 className="text-lg font-semibold mb-4">
    Operaciones Academicas
  </h2>

  {/* Stats de asistencia */}
  <StatsAsistenciaCard apiUrl="/api/admin/asistencias" />

  {/* Links rapidos */}
  <div className="grid grid-cols-3 gap-3 mt-4">
    <a href="/admin/horarios"
      className="flex items-center gap-2 p-3 border rounded-lg
        hover:bg-muted transition-colors">
      <Clock className="h-4 w-4 text-blue-500" />
      <span className="text-sm">Horarios</span>
    </a>
    <a href="/admin/asistencias"
      className="flex items-center gap-2 p-3 border rounded-lg
        hover:bg-muted transition-colors">
      <UserCheck className="h-4 w-4 text-green-500" />
      <span className="text-sm">Asistencias</span>
    </a>
    <a href="/admin/evaluacion"
      className="flex items-center gap-2 p-3 border rounded-lg
        hover:bg-muted transition-colors">
      <BarChart3 className="h-4 w-4 text-purple-500" />
      <span className="text-sm">Evaluacion</span>
    </a>
  </div>
</div>
```

5.2 - Dashboard Docente

TypeScript - Actualizar app/(dashboard)/docente/page.tsx

// Agregar seccion en el JSX:

```
{/* Links rapidos Fase 4 */}
<div className="mt-8">
  <h2 className="text-lg font-semibold mb-4">Accesos Rápidos</h2>
  <div className="grid grid-cols-2 md:grid-cols-4 gap-3">
    <a href="/docente/horarios"
      className="flex flex-col items-center gap-2 p-4 border
        rounded-lg hover:bg-muted transition-colors">
      <Clock className="h-6 w-6 text-blue-500" />
      <span className="text-sm">Mis Horarios</span>
    </a>
    <a href="/docente/asistencias"
      className="flex flex-col items-center gap-2 p-4 border
        rounded-lg hover:bg-muted transition-colors">
      <UserCheck className="h-6 w-6 text-green-500" />
      <span className="text-sm">Pasar Lista</span>
    </a>
    <a href="/docente/evaluacion"
      className="flex flex-col items-center gap-2 p-4 border
        rounded-lg hover:bg-muted transition-colors">
      <BarChart3 className="h-6 w-6 text-purple-500" />
      <span className="text-sm">Ponderaciones</span>
    </a>
    <a href="/docente/notas-finales"
      className="flex flex-col items-center gap-2 p-4 border
        rounded-lg hover:bg-muted transition-colors">
      <Trophy className="h-6 w-6 text-amber-500" />
      <span className="text-sm">Notas Finales</span>
    </a>
  </div>
</div>
```

5.3 - Dashboard Estudiante

TypeScript - Actualizar app/(dashboard)/estudiante/page.tsx

// Agregar seccion en el JSX:

```
{/* Links rapidos Fase 4 */}
<div className="mt-8">
```

```

<h2 className="text-lg font-semibold mb-4">Mi Informacion</h2>
<div className="grid grid-cols-3 gap-3">
  <a href="/estudiante/horarios"
    className="flex flex-col items-center gap-2 p-4 border
      rounded-lg hover:bg-muted transition-colors">
    <Clock className="h-6 w-6 text-blue-500" />
    <span className="text-sm">Mi Horario</span>
  </a>
  <a href="/estudiante/asistencias"
    className="flex flex-col items-center gap-2 p-4 border
      rounded-lg hover:bg-muted transition-colors">
    <UserCheck className="h-6 w-6 text-green-500" />
    <span className="text-sm">Mis Asistencias</span>
  </a>
  <a href="/estudiante/materias"
    className="flex flex-col items-center gap-2 p-4 border
      rounded-lg hover:bg-muted transition-colors">
    <BookOpen className="h-6 w-6 text-purple-500" />
    <span className="text-sm">Mis Materias</span>
  </a>
</div>
</div>

```

Paso 6: Checklist de Cierre - Fase 4 Completa

Verifica que TODOS los componentes de la Fase 4 esten funcionando correctamente antes de marcar la fase como completada.

Base de Datos (Entrega 8A)

- ✓ Tabla horarios creada con CHECK constraints, indices y UNIQUE.
- ✓ Tabla asistencias creada con trigger updated_at y UNIQUE(inscripcion, fecha).
- ✓ Tabla configuracion_evaluacion creada con CHECK peso BETWEEN 0-100.
- ✓ RLS habilitado en las 3 tablas con politicas diferenciadas por rol.
- ✓ Los 3 indices principales funcionan: idx_horarios_materia, idx_asistencias_inscripcion, idx_config_eval_materia.

Modulo Horarios (Entregas 8A-8B)

- ✓ Schema Zod horarioSchema con .refine() para hora_fin > hora_inicio.
- ✓ API CRUD admin: GET, POST, PUT, DELETE con manejo de error 409 (duplicado).
- ✓ API GET docente: solo materias asignadas.
- ✓ API GET estudiante: solo materias inscritas con estado 'inscrito'.
- ✓ Componente HorariosTable con Dialog crear/editar, AlertDialog eliminar, filtro por dia.
- ✓ Componente HorarioSemanal reutilizable con grilla 6 columnas y colores por modalidad.

Modulo Asistencias (Entregas 9A-9B)

- ✓ Schema Zod asistenciaMasivaSchema con validacion de fecha no futura.
- ✓ API POST docente con UPSERT masivo atomico y verificacion de integridad.
- ✓ API PUT docente individual para correcciones en historial.
- ✓ API GET estudiante con estadisticas pre-calculadas por materia.
- ✓ API GET admin con resumen global y estudiantes en riesgo.
- ✓ Componente PasarLista con tabla interactiva, botones 'Todos Presente/Ausente', contadores.
- ✓ Componente HistorialAsistencias con filtros y edicion inline.

- ✓ Componente MisAsistencias (estudiante) con tarjetas expandibles, Progress bar, alerta riesgo.
- ✓ Componente AsistenciasResumen (admin) con 4 tarjetas KPI y tabla por materia.
- ✓ Componente StatsAsistenciaCard reutilizable para dashboards.

Modulo Evaluacion (Entregas 10A-10B)

- ✓ Schema Zod configEvaluacionSchema + funcion verificarSumaPesos.
- ✓ API CRUD admin con suma post-operacion en respuesta.
- ✓ API GET docente agrupado por materia con suma.
- ✓ API GET estudiante con ponderaciones de materias inscritas.
- ✓ API GET docente/notas-finales con calculo de nota final ponderada.
- ✓ Funcion pura calcularNotaFinal con desglose, faltantes y estados.
- ✓ Componente ConfigEvaluacionTable con indicador visual de suma en tiempo real.
- ✓ Componente PonderacionesMateria de solo lectura con barras de peso.
- ✓ Componente NotasFinalesTable con desglose expandible y KPIs.
- ✓ Dashboards actualizados con metricas y links rapidos de Fase 4.

Navegacion y UX

- ✓ Sidebar actualizado con Horarios, Asistencias, Evaluacion para los 3 roles.
- ✓ Notas Finales agregado al sidebar del docente.
- ✓ Todas las paginas cargan correctamente sin errores en consola.
- ✓ npm run dev funciona sin errores.
- ✓ Componentes shadcn/ui requeridos instalados: Progress, Tabs, Switch.

Resumen de Entregas: Fase 4 Completa

Entrega	Titulo	Archivos Nuevos	Lineas Aprox.
8A	Schema SQL, RLS y Navegacion	3 tablas SQL + 12 politicas RLS + 8 paginas placeholder	~200 SQL + 8 TSX
8B	API Routes y UI de Horarios	1 schema + 4 API routes + 2 componentes + 3 paginas	~600 TS/TSX
9A	Asistencias: Registro Docente	1 schema + 3 API routes + 2 componentes + 1 pagina	~700 TS/TSX
9B	Asistencias: Consulta y Stats	2 API routes + 3 componentes + 2 paginas	~500 TS/TSX
10A	Configuracion de Evaluacion	1 schema + 5 API routes + 2 componentes + 2 paginas	~650 TS/TSX
10B	Nota Final, Dashboards, Cierre	1 utilidad + 1 API route + 1 componente + 1 pagina + dashboards	~500 TS/TSX

Inventario Total del Sistema (Fases 1-4)

Componente	Fase 1-3 (MVP)	Fase 4	Total
Tablas en Supabase	5	3	8
Políticas RLS	~15	12	~27
API Routes	~10	15	~25
Schemas Zod	3	3	6
Componentes React	~12	12	~24
Paginas Next.js	~10	10	~20
Funciones utilitarias	2	2	4

TIP: Arquitectura escalable

El sistema sigue un patron consistente en todas las fases: Schema Zod para validacion dual -> API Routes con verificacion de rol -> Componentes React con estado local y fetch -> Paginas Next.js como wrappers delgados. Este patron se puede replicar para agregar nuevos modulos en futuras fases.

Ejercicio Final de la Fase 4

Completa estas 4 tareas integradoras que combinan conocimientos de toda la Fase 4:

1. Tarea 1: Crea un endpoint GET /api/estudiante/notas-finales que devuelva la nota final ponderada del estudiante logueado en cada materia inscrita. Muestra los resultados en una nueva pagina /estudiante/notas con tarjetas similares a MisAsistencias.
2. Tarea 2: Implementa un endpoint GET /api/admin/reporte-general que devuelva un JSON con el resumen completo del sistema: total usuarios por rol, materias con horarios incompletos, materias con config de evaluacion incompleta, y estudiantes en riesgo de asistencia. Esto seria la base para un 'reporte ejecutivo'.
3. Tarea 3: Agrega exportacion PDF al componente NotasFinalesTable. Al hacer click en 'Exportar PDF', genera un documento con la tabla de notas, el resumen KPI y el nombre de la materia. Pista: usa la libreria jsPDF o similar.
4. Tarea 4: Crea un componente CalendarioAsistencias que muestre un mini-calendario mensual con colores por dia (verde = presente, rojo = ausente, amarillo = tardanza/justificado). Integralo en la vista de MisAsistencias del estudiante. Pista: genera un grid 7x5 con los dias del mes.

Cierre y Proximos Pasos

Con la Entrega 10B completamos la Fase 4 del Sistema de Gestion Academica. El sistema ahora tiene 8 tablas, ~25 API Routes, ~24 componentes React y ~20 paginas Next.js, cubriendo la gestion completa de usuarios, materias, inscripciones, calificaciones, documentos, horarios, asistencias y evaluacion.

Posibles fases futuras podrian incluir: notificaciones en tiempo real (Supabase Realtime), generacion de reportes PDF, modulo de mensajeria interna docente-estudiante, integracion con calendario externo (Google Calendar), o un dashboard analitico con graficos avanzados usando Recharts.

NOTA: Felicitaciones

Has construido un sistema completo de gestion academica full-stack con Next.js, Supabase, TypeScript y Tailwind CSS. Cada entrega siguio el patron: teoria -> implementacion -> verificacion -> ejercicio. Este proyecto demuestra competencia en desarrollo web moderno, bases de datos relacionales, seguridad RLS y diseno de interfaces.